PES UNIVERSITY

PROJECT TEST PLAN

Dehallucination Add-On Module for Enhancing Reliability of AI Applications

Jackfruit Phase-1 Mini Project

Item	Details
Team No.	31
Team Name	FANTASTIC FOUR
Problem Statement	39
Document Version	1.0
System Type	Add-on verification module with a REST API and a web UI
Backend	Python + FastAPI
Database	SQLite (PostgreSQL possible later)
Testing Basis	SRS Version 1.0, Software Architecture Document, Jira Backlog (SCRUM, 47 stories)
Testing Scope	Academic proof-of-concept

Team Member	SRN
Kushal J	PES2UG24AM205
Laxmikant	PES2UG24AM206
Gagan K C	PES2UG24AM201
Yashwanth G	PES2UG24AM189

1. Introduction

This Project Test Plan describes how the Dehallucination Add-On Module will be tested. It checks that the implemented system meets the SRS requirements (FR-01 to FR-30, Section 5.10 and Sections 6 and 7) and behaves as described in the Software Architecture Document. Every test case is traced to an SRS requirement and to a Jira story (SCRUM key).

Values the SRS leaves open, such as the response-time target and the claim-status thresholds, use the proposed values of the Software Architecture Document (Section 5.5), to be agreed with the project guide.

2. Testing Objectives

- Verify that the functional requirements of the SRS are implemented correctly.
- Verify authentication (web login and API keys) and role-based authorization for End User, Developer/Integrator, Reviewer and Administrator.
- Verify input validation, request ID handling and preprocessing with correct claim offsets.
- Verify claim extraction, including text with no verifiable claim.
- Verify evidence retrieval, source failures, timeouts, bounded retries and caching.
- Verify that claims are classified as Supported, Contradicted or Uncertain, and that an Uncertain claim is never presented as verified.
- Verify confidence scores, the review flag, marked and final responses and suggested statements.
- Verify the verification API, its structured result and its error responses.
- Verify trusted-source management, the audit log and the reviewer list.
- Verify the web UI, security, usability, reliability, safety and basic performance expectations.

3. Scope of Testing

3.1 In Scope

- Input validation, request IDs and preprocessing
- Claim extraction with offsets
- Evidence retrieval, source connectors, retries and caching
- Evidence-quality rules, NLI verification, confidence scoring and the review flag
- Response processing: marks, marked response, final response, suggested statement and overall status
- Pipeline failure handling and the verification API
- Authentication and role-based access control
- Trusted-source management and configuration
- Audit log, reviewer list and persistence, including persistence failure
- Web UI pages
- Security, usability, reliability, safety, compatibility and basic performance

3.2 Out of Scope

- The accuracy and internal behaviour of the AI model that generated the original text (the module is an add-on, SRS 1.3).
- The correctness and completeness of Wikipedia, Wikidata or other third-party sources; only the module's handling of their data is tested.
- Large-scale load testing and production deployment; running several Uvicorn workers.
- Domain-specific validation for safety-critical use (SRS 6.2).
- Automatic generation of corrected text with a language model (not part of the design).

- Migration from SQLite to PostgreSQL; browsers other than Chrome beyond a sanity check in Firefox and Edge.

4. Test Strategy

Testing is performed progressively from single components to the complete system. Test cases are black-box and derived from the SRS requirements, the architecture and the Jira stories.

Test Level / Type	Purpose
Unit Testing	Verify single components one at a time with pytest, without starting the web server.
Integration Testing	Verify the pipeline between components and with the database, using mock trusted sources.
System / API Testing	Verify the complete flow through the REST API and the web UI (Postman and browser).
Functional Testing	Verify each functional requirement and its expected behaviour (Section 7).
Verification-Quality Testing	Run the labelled claim set (10 supported, 10 contradicted, 10 unverifiable claims) and compare the statuses with the expected ones.
Security Testing	Verify authentication, API keys, password and key hashing, role-based access and access to results and logs.
Performance Testing	Measure response time against the proposed target and check caching and graceful degradation (Locust or JMeter).
Reliability / Fault Testing	Inject source failures, timeouts, a database failure and a model failure; check controlled handling.
Usability Testing	Check that representative users understand the statuses, the confidence label and the evidence.
Compatibility Testing	Check the web UI core flow in Chrome, with a sanity check in Firefox and Edge.
Regression Testing	Re-run the affected tests, and all High-priority tests, after fixes or changes.

5. Test Environment

Area	Planned Environment
Client	Desktop or laptop (Windows or Linux) with Google Chrome; Firefox and Edge for a sanity check; Postman as API client
Backend	Python + FastAPI served by Uvicorn (single worker)
Database	SQLite test database with the 8-table schema
NLP / AI	spaCy for claim extraction; a pretrained NLI model (Hugging Face Transformers) for verification
Trusted Sources	Mock sources for repeatable tests; Wikipedia and Wikidata APIs for live sanity runs
Tools	pytest, Postman, Locust or JMeter, Git, Jira (space SCRUM)
Network	Local environment; internet only for live-source runs
Test Data	Synthetic data (Section 10) and test accounts: End User, Reviewer, Administrator and an API client

6. Entry and Exit Criteria

Criteria	Definition
Entry Criteria	SRS, architecture and test cases are available; the build starts; the test environment is configured (models loaded, test database, mock sources); test data and test accounts are prepared.

Criteria	Definition
Exit Criteria	All planned test cases are executed; 100% of High-priority test cases pass; at least 90% of Medium and Low priority test cases pass, and failures have an agreed workaround; no open Critical or High severity defects; the full pipeline test (TC-039) and the safety test (NFR-004) pass; the response-time target in NFR-001 is measured.
Suspension Criteria	Testing stops if the build does not start, the API does not respond, or a Critical defect blocks more than 30% of the planned test cases.
Resumption Criteria	The blocking defect is fixed and the smoke tests (TC-001, TC-009, TC-039) pass.
Blocked Test	A test cannot be executed because a required dependency or environment component is unavailable.
Pass	The actual result satisfies the expected result.
Fail	The actual result does not satisfy the expected result.

7. Functional Test Cases

This plan has 58 functional test cases (49 High, 9 Medium, 0 Low priority). The Traceability column gives the SRS requirement (FR number, SRS section or Appendix B) and the Jira story with its SCRUM key.

ID	Area	Scenario	Test Steps / Input	Expected Result	Traceability	Priority
TC-001	Startup	Application starts and reports health	Start the app with Uvicorn; call GET /health; send two verification requests and check the application log	Health check returns OK with the model status and no sensitive details; the models are loaded once at start-up, not for every request	Sec 6.1, Sec 6.4; US-001 (SCRUM-17), US-004 (SCRUM-20)	High
TC-002	Configuration	Configuration values load and invalid values are rejected	Set thresholds, timeout, retry limit and claim limit in the config file and start the app; then put an invalid value (for example a negative timeout) in the file and start again	Configured values are used and the defaults match SAD Section 5.5; an invalid value gives a clear error and is not used	FR-18, Sec 7; US-002 (SCRUM-19)	Medium
TC-003	Database	Schema and constraints	Initialise the database; then try to insert a claim with status 'Maybe', a claim with confidence 1.5, a second suggested statement for one claim, and a request with both user_id and client_id	The 8 tables exist (USER, API_CLIENT, VERIFICATION_REQUEST, CLAIM, EVIDENCE, TRUSTED_SOURCE, CORRECTED_RESPONSE, AUDIT_LOG); all four invalid inserts are rejected	Sec 7, App B, FR-13, FR-16; US-003 (SCRUM-18)	High
TC-004	Authentication	Web login (valid and invalid)	Log in with a correct username and password; then log in with a wrong password	A correct login returns a token and protected pages open; a wrong login returns HTTP 401 and no token	Sec 6.3; US-005 (SCRUM-21), US-042 (SCRUM-57)	High
TC-005	Authentication	Passwords are stored as hashes	Register a test user; inspect the USER table	No plain-text password is stored	Sec 6.3; US-005 (SCRUM-21)	High
TC-006	Authentication	API key authentication	Call POST /api/v1/verify with a valid key, with no key, with a wrong key, and with the key of a disabled API client	The valid key is accepted; a missing or wrong key returns HTTP 401; the disabled client is rejected	Sec 6.3, FR-28; US-006 (SCRUM-22)	High
TC-007	Authorization	Source changes are refused to non-administrators	As an End User call POST /api/v1/sources; then, without credentials, call PUT /api/v1/sources/{source_id}	HTTP 403 for the End User and HTTP 401 without credentials; the source configuration is unchanged	FR-30, Sec 6.5; US-007 (SCRUM-23), US-034 (SCRUM-49)	High
TC-008	Authorization	Role rules match the design	Call each endpoint with each role (End User, Developer/Integrator, Reviewer, Administrator)	Every call is allowed or refused exactly as in the SAD role-permission table; refused calls return 403	FR-30, Sec 6.5; US-007 (SCRUM-23)	High
TC-009	Input	Valid text is accepted	Send POST /verify with {"response": "The Eiffel Tower is in Paris."}	The request is accepted, processing starts and a request ID is returned	FR-01; US-008 (SCRUM-24), US-028 (SCRUM-43)	High
TC-010	Input	Invalid requests are rejected	Send: a broken JSON body; {} (no response field); an empty text; a whitespace-only text; and "response": 123	Broken JSON and the missing field return HTTP 400; the empty text, the whitespace-only text and the non-text value return HTTP 422; each gives a clear error message and no verification is performed	FR-02, FR-03; US-008 (SCRUM-24)	High

ID	Area	Scenario	Test Steps / Input	Expected Result	Traceability	Priority
TC-011	Request ID	Request ID handling	Send a request without request_id; send 100 such requests; send a request with request_id "REQ-100"; send "REQ-100" again	A unique ID is generated and returned when it is omitted (all 100 IDs differ); a supplied ID is returned and the result is stored under it; the duplicate is rejected with HTTP 422	FR-01, Sec 3.3, App B; US-008 (SCRUM-24)	High
TC-012	Input	Long text and special characters	Send a response of about 2000 words; then send text with accents, symbols and emoji	Both are processed without truncation, error or crash	FR-01, FR-04; US-008 (SCRUM-24), US-009 (SCRUM-25), US-010 (SCRUM-26)	Medium
TC-013	Preprocessing	Preprocessing keeps offsets correct	Send text with extra spaces, tabs and stray symbols before a factual sentence	The cleaned text is normalised, the original is unchanged, and the claim's start and end offsets select the correct sentence in the ORIGINAL text	FR-04, FR-06; US-009 (SCRUM-25), US-011 (SCRUM-27)	High
TC-014	Claim extraction	Claims are extracted; opinions are ignored	Send one factual sentence; then a paragraph with three facts; then "I think blue is nice. Is it raining? Hello!"	One fact gives one claim; three facts give three claims; the opinion, the question and the greeting give no claim	FR-05; US-010 (SCRUM-26)	High
TC-015	Claim extraction	Claim IDs, request link and offsets	Send text with two claims; inspect the result; compare original_response[start:end] with claim_text for each claim	Each claim has a claim ID and is linked to its request; the text at the offsets equals the claim text (zero-based, end excluded)	FR-06; US-011 (SCRUM-27)	High
TC-016	Claim extraction	No verifiable claim	Send "Hello! How can I help you today?"	Overall status is No Verifiable Claims; retrieval and NLI are skipped; no error is returned	FR-07; US-012 (SCRUM-28)	High
TC-017	Retrieval	Connector interface with the mock source	Query the mock connector with a claim	Fixed evidence is returned with its source reference; a new connector needs no change in the coordinator	FR-09; US-013 (SCRUM-29)	High
TC-018	Retrieval	Wikipedia and Wikidata connectors (live sanity run)	Query the Wikipedia connector with "The Eiffel Tower is in Paris." and the Wikidata connector with a known entity fact	Relevant snippets or facts are returned with their reference; an empty result is handled; errors do not crash the system	FR-09, FR-10; US-014 (SCRUM-30), US-015 (SCRUM-31)	Medium
TC-019	Retrieval	One query per claim, enabled sources only	Send a response with three claims; enable source A and disable source B; check the generated queries and the source calls	Three queries are created, one per claim, and only source A is queried	FR-08, FR-09, Sec 6.5; US-016 (SCRUM-32), US-033 (SCRUM-48)	High
TC-020	Retrieval	Source information is recorded with evidence	Verify a claim; inspect the evidence record	The source and page reference are stored with every evidence item	FR-10; US-017 (SCRUM-33)	High
TC-021	Retrieval	Source unavailable or no relevant evidence	Make the mock source return HTTP 503 and verify a claim; then verify an unknown claim against a source that returns nothing	After one retry the claim is Uncertain with the reason 'retrieval failure' (first case) or 'no evidence' (second case); it is never Supported and the result says verification could not be completed for it	FR-11, Sec 6.2; US-016 (SCRUM-32)	High
TC-022	Retrieval	Timeout and bounded retries	Delay the mock source by more than 5 seconds; then make a source fail on every call and count the calls for one claim	The timeout is recorded and the system does not hang; the source is called once plus one retry only	FR-11, Sec 6.1; US-016 (SCRUM-32)	High
TC-023	Verification	Weak evidence is filtered out	Provide a snippet with relevance below 0.5; then verify "It was built in 1950." with a snippet that does not contain the year	Neither snippet qualifies; with no qualifying snippet the claim is Uncertain, not Supported	FR-12, Sec 2.5; US-018 (SCRUM-34)	High

ID	Area	Scenario	Test Steps / Input	Expected Result	Traceability	Priority
TC-024	Verification	Supported and Contradicted claims	Use a claim with clearly entailing evidence (E at least 0.70, C below 0.30) and a claim with clearly contradicting evidence (C at least 0.70, E below 0.30)	The first claim is Supported; the second is Contradicted	FR-13; US-019 (SCRUM-35)	High
TC-025	Verification	Uncertain claims	Use a claim with neutral evidence; a claim with one supporting and one contradicting source; a claim whose evidence neither supports nor contradicts it	All three are Uncertain, never Supported; for conflicting evidence both evidence items are kept	FR-12, FR-13, FR-15, Sec 2.5; US-019 (SCRUM-35)	High
TC-026	Verification	High NLI score on weak evidence	Provide off-topic or low-quality evidence that still gets a high entailment score	The claim is not Supported (the evidence-quality check applies)	Sec 2.5, Sec 6.5; US-018 (SCRUM-34), US-019 (SCRUM-35)	High
TC-027	Verification	Evidence is preserved for each decision	Verify claims; inspect the results and the database	The evidence used for each status is stored and returned with the claim, linked by request, claim and evidence IDs	FR-14, Sec 6.4; US-020 (SCRUM-36)	High
TC-028	Verification	Uncertain claim is never marked verified	Verify a claim that ends Uncertain; inspect the claim and the overall status	The claim is not shown as verified and the overall status is not Verified	FR-15, Sec 6.2; US-019 (SCRUM-35), US-025 (SCRUM-40)	High
TC-029	Confidence	Confidence is assigned and linked	Verify three claims	Each claim has a confidence value between 0 and 1 linked to its claim ID	FR-16, FR-17; US-021 (SCRUM-37)	High
TC-030	Confidence	Review flag rule	Verify an Uncertain claim; a Supported claim with confidence below 0.6; a Contradicted claim with confidence at or above 0.6	review_flag is true for the first two; for the Contradicted claim it is false, but the content is still marked in the response	FR-18, FR-20; US-021 (SCRUM-37), US-023 (SCRUM-38)	High
TC-031	Confidence	Threshold comes from configuration	Change the threshold from 0.6 to 0.9; re-verify a claim with confidence 0.8	The claim is now flagged; no code change is needed	FR-18, Sec 7; US-021 (SCRUM-37), US-002 (SCRUM-19)	Medium
TC-032	Response	Contradicted and Uncertain content is found and marked	Verify "The Sun orbits the Earth.", an unverifiable claim and a fully correct sentence	The contradicted sentence is located through its offsets and marked Contradicted; the unverifiable sentence is marked unverified; the correct sentence is unchanged	FR-19, FR-20, FR-21; US-023 (SCRUM-38), US-024 (SCRUM-39)	High
TC-033	Response	Mark is placed after the right claim	Verify a response with a contradicted claim in the middle of several sentences	The mark appears directly after that claim, not at the end of the response	FR-20, FR-21; US-024 (SCRUM-39)	High
TC-034	Response	Final response and suggested statement	Verify a response with one contradicted claim; inspect final_response and suggested_statement for a Contradicted and a Supported claim	final_response contains the marked text with the suggested statement beside the mark; the suggested statement is a sentence taken from the evidence and is null for a Supported claim or when no suitable sentence exists; no text-generation model is used	FR-21; US-024 (SCRUM-39)	High
TC-035	Response	Overall status values	Verify: all claims Supported; one claim Contradicted; a text with no claims	Verified (final_response equals the original), Partially Verified/Flagged and No Verifiable Claims respectively	FR-21, FR-23; US-024 (SCRUM-39), US-025 (SCRUM-40)	High
TC-036	Response	The three statuses are distinguishable	Verify a response with one claim of each status	Supported, Contradicted and Uncertain are clearly different in the result	FR-25, Sec 6.5; US-025 (SCRUM-40)	High

ID	Area	Scenario	Test Steps / Input	Expected Result	Traceability	Priority
TC-037	Response	Evidence and sources are attached	Open the result of a verified claim	Evidence text and source reference are returned; if no evidence exists the result says so	FR-24; US-025 (SCRUM-40)	High
TC-038	Response	Original response is retained	Verify with logging on and inspect the stored record; then switch logging off, verify and inspect again	With logging on the original text is stored unchanged next to the result; with logging off it is not stored	FR-22; US-026 (SCRUM-41)	Medium
TC-039	Pipeline	Full pipeline on a mixed response	Send a response with one true, one false and one unverifiable claim	Statuses are Supported, Contradicted and Uncertain; the response is marked; evidence is attached; the record is stored	FR-12, FR-13, FR-14, FR-15; US-027 (SCRUM-42), US-028 (SCRUM-43)	High
TC-040	API	Response contains all structured fields	Call POST /verify; check the JSON	It contains request_id, overall_status, claims (offsets, status, confidence, review_flag, evidence, suggested_statement), original_response, marked_response, final_response, stored, unprocessed_claim_count and timestamp	FR-26, FR-27; US-028 (SCRUM-43)	High
TC-041	API	Error responses	Trigger an invalid format, a missing key, no permission, an unknown ID and invalid input	HTTP 400, 401, 403, 404 and 422 are returned, each with an error code and a message	FR-28; US-029 (SCRUM-44)	High
TC-042	API	Service unavailable	Make the NLI model unavailable; call POST /verify	HTTP 503 with an error message; no fake success result	FR-28, Sec 3.3; US-029 (SCRUM-44), US-027 (SCRUM-42)	High
TC-043	Pipeline	System failure in the middle of verification	Stop the NLI model after the first claim is verified	Results completed so far are returned; overall status is Verification Incomplete; unprocessed_claim_count is above 0; the response is never shown as Verified	Sec 3.3, Sec 6.4; US-027 (SCRUM-42)	High
TC-044	Pipeline	Missing evidence is not an incomplete verification	Make all sources fail for one claim	The claim is Uncertain with a reason; the overall status is Partially Verified/Flagged, not Verification Incomplete	FR-11, Sec 6.4; US-016 (SCRUM-32), US-027 (SCRUM-42)	High
TC-045	Results	Owner and other users	Verify a text; later call GET /results/{request_id} as the same user; then as a different End User	The owner gets the stored result with "stored": true; the other user is refused (403 or 404)	FR-23, Sec 6.5, Sec 7; US-030 (SCRUM-45), US-031 (SCRUM-46)	High
TC-046	Results	Reviewer and Administrator access	As a Reviewer fetch a result with a flagged claim and one without; as an Administrator fetch any result; call GET /results as each role	The Reviewer gets only the result with a flagged claim; the Administrator gets any result; the list shows each role only what it may fetch	Sec 6.5, Sec 5.10; US-030 (SCRUM-45)	High
TC-047	Persistence	Database unavailable while saving	Make the database unavailable after verification completes; verify a text	The completed result is still returned with "stored": false; the failure and the event are written to the application log file; the response does not imply it was stored	Sec 6.4, Sec 5.10; US-031 (SCRUM-46), US-035 (SCRUM-50)	High
TC-048	Sources	Administrator manages sources	As an Administrator list, add and update a source; then add a source with an empty or invalid URL	The sources are listed; a valid source is saved as enabled and an update is used by retrieval; the invalid source gives a validation error and nothing is saved	FR-29; US-032 (SCRUM-47)	High

ID	Area	Scenario	Test Steps / Input	Expected Result	Traceability	Priority
TC-049	Sources	Disable and remove sources	Disable a source and verify a claim; remove a source already used as evidence; remove a never-used source	The disabled source is never queried; the used source is only disabled and its old evidence stays valid; the unused source is deleted	FR-29, Sec 6.5; US-033 (SCRUM-48)	High
TC-050	Logging	Requests, results and errors are logged	Verify a text; cause a retrieval failure; inspect the audit log	Entries with the request, a result summary and a timestamp exist; the failure has an entry with an error code	Sec 5.10, App B; US-035 (SCRUM-50)	High
TC-051	Review	Reviewer list shows flagged and uncertain results	As a Reviewer, call GET /review/flagged	Claims with a review flag or status Uncertain are listed; other records are not visible	Sec 5.10, Sec 6.5; US-036 (SCRUM-51)	High
TC-052	Review	Audit log access	As an Administrator call GET /logs; as an End User call GET /logs	The Administrator sees the log entries; the End User gets HTTP 403	Sec 5.10, Sec 6.5; US-037 (SCRUM-52), US-007 (SCRUM-23)	Medium
TC-053	Logging	User text is minimised in logs	Submit text containing a fake email address and phone number; inspect the log	Sensitive details are masked or not stored beyond what is needed	Sec 6.3; US-037 (SCRUM-52)	Medium
TC-054	Web UI	Login page	Open the login page; log in correctly, then incorrectly	A correct login opens the verify page; a wrong login shows a clear error message	Sec 3.1; US-042 (SCRUM-57)	High
TC-055	Web UI	Verify page and error messages	Paste text and click Verify; then click Verify with empty text; then stop the API and click Verify	The overall status is displayed; a clear error message is shown for the empty text and for the unavailable API	Sec 3.1, FR-23; US-038 (SCRUM-53)	High
TC-056	Web UI	Claim-level results and responses	Verify a mixed response that includes a contradicted claim	Each claim shows status, confidence (labelled as a model score), evidence and source; the three statuses look different; the marked response and the final response (with the suggested statement) are shown	FR-20, FR-21, FR-23, FR-24, FR-25; US-039 (SCRUM-54)	High
TC-057	Web UI	Source management page	As an Administrator list, add, edit, disable and remove a source; then open the page as an End User	The actions work for the Administrator; the page is not available to other roles	FR-29, FR-30; US-040 (SCRUM-55)	Medium
TC-058	Web UI	Reviewer page	As a Reviewer, open the review page	Flagged and uncertain claims are listed with their evidence	Sec 5.10; US-041 (SCRUM-56)	Medium

8. Non-Functional Test Plan

Performance values are the proposed targets from the Software Architecture Document (Section 5.5). The SRS leaves the response-time target open.

ID	Type	Test	Expected Result	Reference	Priority
NFR-001	Performance	Send 100 requests with 5 claims each against mock sources and measure the time	At least 95% finish within 10 seconds (proposed target; SRS 6.1 leaves the value open)	Sec 6.1; US-046 (SCRUM-61)	Medium
NFR-002	Performance	Delay one source by 8 seconds while verifying a response with 5 claims; then verify the same claim repeatedly	The slow source is reported, the rest of the result is still returned and the system does not hang (graceful degradation); repeated claims are served from the cache with fewer source calls	Sec 6.1; US-016 (SCRUM-32), US-017 (SCRUM-33), US-046 (SCRUM-61)	Medium
NFR-003	Reliability	Stop the sources one by one, then all of them	No crash; the affected claims become Uncertain; no failure is reported as success	Sec 6.4, Sec 3.3; US-016 (SCRUM-32), US-045 (SCRUM-60)	High
NFR-004	Safety	Run the labelled claim set and check every Uncertain claim; then remove all trusted evidence and verify a text	No Uncertain claim is presented as a confirmed fact or marked Supported; with no evidence the result states that verification could not be completed	Sec 6.2, FR-15; US-016 (SCRUM-32), US-043 (SCRUM-58)	High
NFR-005	Verification quality	Run the 30 labelled claims (10 supported, 10 contradicted, 10 unverifiable) with mock evidence and compare with the expected statuses	The match rate per status is recorded; no unsupported claim is marked Supported; the target match rate is agreed with the guide (not an SRS requirement)	Sec 6.4; US-043 (SCRUM-58)	Medium
NFR-006	Security	Call every endpoint with every role and with no credentials	Unauthorised calls are refused (401 or 403) and nothing changes	Sec 6.3, Sec 6.5; US-007 (SCRUM-23), US-046 (SCRUM-61)	High
NFR-007	Security	Check passwords and API keys in the database and the log files; access the deployed service over HTTPS and over plain HTTP	Only hashes are stored and logs avoid unnecessary user text; HTTPS works and plain HTTP is refused or redirected in the deployed setup	Sec 6.3; US-037 (SCRUM-52), US-046 (SCRUM-61)	High
NFR-008	Usability	Ask 3 representative users to verify a response and explain a result	At least 2 of 3 identify each claim's status, evidence and source correctly	Sec 6.4; US-038 (SCRUM-53), US-039 (SCRUM-54)	Medium
NFR-009	Maintainability and testability	Run the unit tests of each pipeline component (validation, extraction, retrieval, verification, response processing) one at a time without the web server	Each stage can be tested independently	Sec 6.4; US-044 (SCRUM-59)	Medium
NFR-010	Interoperability	Call the API from Postman and from a small sample client script	Both receive the same structured JSON result	Sec 6.4; US-045 (SCRUM-60)	Medium
NFR-011	Compatibility and portability	Run the web UI core flow in Chrome with a sanity check in Firefox and Edge; install and run the application on Windows and on Linux	The core flow works in Chrome with no blocking defect in the other browsers; the application starts and passes the smoke tests (TC-001, TC-009, TC-039) on both operating systems	Sec 2.4, Sec 6.4; US-001 (SCRUM-17), US-038 (SCRUM-53), US-047 (SCRUM-62)	Low

9. Defect Management

Defects found during testing are recorded in the project's Jira space (SCRUM) as a work item with the label "defect", created as a Subtask of the affected story. Each defect contains a clear title, description, steps to reproduce, expected result, actual result, severity, priority, environment, evidence, and the linked test case ID and story key.

Severity	Meaning	Example
Critical	A core function or a major security or safety rule fails; testing may be blocked.	A claim without evidence, or an Uncertain claim, is shown as Supported or Verified; a non-administrator can change trusted sources; the application cannot start.
High	An important requirement fails.	Claim offsets point to the wrong text; a retrieval failure is not reported; a result is returned for the wrong user.
Medium	Important but non-blocking issue.	The confidence is not labelled as a model score in the UI; a log entry is missing its error code.
Low	Minor issue with limited impact.	A wording or layout problem in the UI.

Workflow: Open, In Progress, Fixed, Retest, Closed (or Reopened). The tester of the failed test retests the fix, and the High-priority tests are re-run as regression after each fix.

10. Test Data

- Labelled claim set of 30 claims: 10 supported, 10 contradicted and 10 unverifiable, each with its expected status (Jira story US-043).
- Mock source responses: relevant evidence, neutral evidence, contradicting evidence, conflicting evidence, no result, HTTP 503 and a delayed response.
- Sample AI responses: one fact, several facts, opinions only, greeting only, a mixed true/false/unverifiable response, a 2000-word text, text with extra whitespace, and text with Unicode characters.
- Requests with valid and invalid fields: empty text, whitespace-only text, missing field, malformed JSON, non-text value, omitted request ID, supplied request ID and duplicate request ID.
- Test accounts and credentials: End User, Reviewer, Administrator, an active API client and a disabled API client.
- Text containing fake personal data (email address, phone number) for the log-masking test.
- Configuration variants: default thresholds and changed thresholds (for example 0.6 and 0.9).

11. Risks and Mitigation

Risk	Impact	Mitigation
Wikipedia or Wikidata unavailable or rate-limited	Live-source tests fail or slow down	Use mock sources for functional tests; run live sources only for sanity checks; use caching.
NLI model gives an unstable or wrong result	Wrong claim status	Fix the model version; use the labelled claim set; tune the proposed thresholds; keep the evidence-quality rules.
No ground truth beyond the labelled claim set	Accuracy cannot be judged	Build the labelled claim set (US-043) and agree the target match rate with the guide.
Limited source coverage	Many claims end Uncertain	This is allowed by the SRS and safer than guessing; use mock evidence for status tests.
Slow model start-up or low memory	Slow or failing tests	Load models once at start-up; use a smaller model on low-memory machines.

Risk	Impact	Mitigation
SRS values left open (response time, field lengths)	Performance tests cannot pass or fail	Use the proposed values of the architecture document and confirm them with the guide.
Database failure while saving	Results not stored	Test the fallback: result returned with stored false and the failure written to the application log file.
Unauthorized access	Security or privacy problem	Test every endpoint with every role and with no credentials (TC-008, NFR-006).
Limited team time	Lower coverage	Run High-priority tests first; automate with pytest; re-run High-priority tests as regression.

12. Traceability Summary

The test plan traces the SRS requirement groups and the Jira epics to executable tests. The final implementation should update the actual result, status, execution date, tester and defect ID for each test.

12.1 SRS Requirements to Test Cases

Requirement Group	Area	Test Coverage
FR-01 to FR-04	Response Input and Validation	TC-009 to TC-013
FR-05 to FR-07	Claim Extraction	TC-013 to TC-016
FR-08 to FR-11	Trusted Evidence Retrieval	TC-017 to TC-022, TC-044
FR-12 to FR-15	Fact Verification	TC-003, TC-023 to TC-025, TC-027, TC-028, TC-039
FR-16 to FR-18	Confidence Scoring	TC-002, TC-003, TC-029 to TC-031
FR-19 to FR-22	Hallucination Detection and Response Processing	TC-030, TC-032 to TC-035, TC-038, TC-056
FR-23 to FR-25	Evidence and Result Presentation	TC-035 to TC-037, TC-045, TC-055, TC-056
FR-26 to FR-28	Verification API	TC-006, TC-040 to TC-042
FR-29 to FR-30	Trusted Source Management	TC-007, TC-008, TC-048, TC-049, TC-057
Section 5.10	Logging and Review (no FR numbers in the SRS)	TC-046, TC-047, TC-050 to TC-052, TC-058
Section 6 and 7	Non-functional requirements and other requirements	NFR-001 to NFR-011

12.2 Jira Epics to Test Cases

Jira Epic	Key	Stories	Test Coverage
1. Project Setup and Foundation	SCRUM-5	4	TC-001 to TC-003, TC-031; NFR-011
2. Authentication and Access Control	SCRUM-6	3	TC-004 to TC-008, TC-052; NFR-006
3. Input Validation and Preprocessing	SCRUM-7	2	TC-009 to TC-013
4. Claim Extraction	SCRUM-8	3	TC-012 to TC-016
5. Evidence Retrieval and Source Connectors	SCRUM-9	5	TC-017 to TC-022, TC-044; NFR-002 to NFR-004
6. Fact Verification and Confidence	SCRUM-10	4	TC-023 to TC-031
7. Response and Report Generation	SCRUM-11	4	TC-028, TC-030, TC-032 to TC-038
8. Verification Orchestration and API	SCRUM-12	5	TC-009, TC-039 to TC-047
9. Trusted Source Management	SCRUM-13	3	TC-007, TC-019, TC-048, TC-049
10. Logging and Review	SCRUM-14	3	TC-047, TC-050 to TC-053; NFR-007
11. Web UI	SCRUM-15	5	TC-004, TC-054 to TC-058; NFR-008, NFR-011
12. Testing, Documentation and Demo	SCRUM-16	6	NFR-001 to NFR-007, NFR-009 to NFR-011; Section 13 (execution record, US-048)

13. Test Execution Record Template

During actual testing, use the following fields to record execution results.

Test ID	Execution Date	Tester	Actual Result	Status	Defect ID	Remarks
TC-___	DD-MM-YYYY	Name	Observed result	Pass/Fail/Blocked	SCRUM-___	Notes
TC-___	DD-MM-YYYY	Name	Observed result	Pass/Fail/Blocked	SCRUM-___	Notes
TC-___	DD-MM-YYYY	Name	Observed result	Pass/Fail/Blocked	SCRUM-___	Notes

14. Conclusion

This test plan provides a structured approach for validating the Dehallucination Add-On Module. It covers the functional requirements FR-01 to FR-30, the logging and review feature of SRS Section 5.10, and the non-functional requirements of SRS Sections 6 and 7, with traceability from the SRS and the Jira stories to executable tests. The plan should be updated during implementation as actual results and Jira defect records become available.